



UNIVERSIDAD DE BUENOS AIRES
FACULTAD DE CIENCIAS EXACTAS Y NATURALES
FACULTAD DE INGENIERÍA

Algoritmo de entrenamiento de árboles de decisión basados en False Discovery Rate aplicado sobre Gradient Boosting

Joaquín I. Aramendía

Director: Gustavo Denicolay

Buenos Aires, febrero 2025

Índice general

1..	Introducción	1
1.1.	Descripción del problema y motivación	1
1.2.	Trabajos previos	1
1.3.	Objetivos	3
2..	Materiales y Métodos	4
2.1.	Construcción de árboles de decisión en LightGBM	4
2.2.	Métodos	5
2.3.	Datos	6
3..	Experimentos preliminares y resultados	8
4..	Tiempo estimado de trabajo	10

1. INTRODUCCIÓN

1.1. Descripción del problema y motivación

Los árboles de decisión se entrenan sobre datos de observaciones de un fenómeno para intentar predecir la respuesta de un sistema ante un nuevo caso del mismo. Este entrenamiento se realiza con la aplicación de funciones que maximizan ganancia de información (o minimizan error) para la selección de la frontera de decisión.

En general los árboles se utilizan en modelos de ensamble para mejorar su potencial predictivo. Tal es el caso del método de Gradient Boosting que ensambla aditivamente la predicción de cada árbol con los anteriores. Cada árbol de este ensamble trata de predecir los errores (o gradientes) que comete el modelo anterior. Un caso típico de aplicación es el paquete LightGBM[1] que implementa optimizaciones para este tipo de ensambles[2].

Estos modelos se entrenan sobre un conjunto de datos etiquetados que representen casos del proceso generador. Muy frecuentemente el modelo aprende de manera que es capaz de predecir muy bien sobre los datos de entrenamiento o incluso sobre un conjunto de validación pero no resulta la misma performance en datos nuevos, fenómeno conocido como “overfitting” o sobreajuste.

Este trabajo pretende abordar una de las causas del “overfitting” en conjuntos de datos de tamaño considerable donde el fenómeno es dominado por la inferencia múltiple. Se introducirán criterios de crecimiento de árboles con ayuda de datos ficticios (denominados “fakes”) contra los que se comparará cada una de las decisiones que tome el árbol para separar la clase. El objetivo final es proveer de inteligencia a cada decisión de separación intentando identificar variables que sean realmente significativas y descartando las correlaciones espúreas.

1.2. Trabajos previos

Con el advenimiento de los algoritmos y técnicas de estudio masivas y automatizadas, el paradigma de investigación cambia del clásico método hipotético-deductivo a un método de inferencia sobre datos ya dados. Es práctica común que se hagan preguntas sobre el fenómeno que los generó luego de haber recolectado datos en grandes cantidades. En tal ambiente la cantidad de hipótesis a probar es creciente y en general las preguntas que se intenta contestar son en muchos casos sesgadas por los datos que se tiene[3].

Fuera de las aplicaciones con datos no estructurados, donde las redes neuronales profundas son los algoritmos de elección, los métodos basados en árboles de decisión son los mejores algoritmos en tareas de predicción[4]. Estos en general tienden a separar los datos hasta llegar a hojas “puras” (una única clase en cada hoja). En esos casos no se llega a obtener buenos resultados en datos nuevos. Se dice que el árbol tiende al “overfitting” (sobreajustado) sobre los datos de entrenamiento[5].

Varias técnicas se utilizan para moderar su crecimiento expresadas en hiperparámetros de los algoritmos y técnicas de separación de los conjuntos de datos: validación cruzada, entrenamiento y prueba o combinaciones de ambos. Estas técnicas tienden a cuantificar el “overfitting” o de alguna manera controlarlo para dotar de cierta inteligencia al algoritmo al producir un mejor modelo predictivo.

Una de las formas de “overfitting” podría expresarse como un problema de replicabilidad del experimento. Un árbol entrenado arroja muy buenos resultados en los datos actuales pero no predice de la misma manera en datos nuevos. Este problema es descrito en el trabajo de Ioannidis (2005)[6] de selección de variables significativas para publicación sobre una cantidad muy grande experimentos, también conocido como “sesgo de publicación”. En tal situación en general se pasa por alto el problema de inferencia múltiple y se eligen valores de significancia muy favorables a descubrimientos falsos. Es decir, si se prueban grandes cantidades de correlaciones con un nivel de significancia se espera que, sin correcciones, algunos de esos descubrimientos no sean válidos.

El problema de inferencia múltiple y comparaciones múltiples en algoritmos de inducción, como son la mayoría de los algoritmos de aprendizaje automático, está descrito en el trabajo de David D. Jensen y Paul R. Cohen (2000)[7]. A la vez que describe las técnicas clásicas de prevención del “overfitting”. Análogamente con los descubrimientos publicados, los algoritmos de inferencia cometen los mismos errores. En particular los árboles de decisión ordenan para cada nodo (“hoja”) las variables por su correlación con la clase a predecir.

En ese ordenamiento se establece una métrica de ganancia de información o de correlación con la predicción, y de todas las correlaciones posibles se elige la mayor para dividir el nodo en nuevas hojas. Esto establece una división del espacio y una frontera de decisión nueva. La relación es directa con el problema de inferencia múltiple: muchas variables se correlacionan con la clase a predecir, tantas como variables se tenga en el conjunto de datos. Cada variable que se elige como variable de corte es un descubrimiento de la misma manera que lo puede ser una prueba de hipótesis: tiene un nivel de significancia y por tanto una probabilidad de ser falsa. En las hojas más cercanas a la base del árbol estas correlaciones serán muy fuertes y las decisiones serán verdaderos descubrimientos. En la medida que el árbol crece, estas correlaciones serán cada vez más débiles aumentando la posibilidad de descubrimientos falsos.

En este trabajo se introduce el concepto de *Tasa de Falso Descubrimiento* (“False Discovery Rate” o FDR) durante el entrenamiento de árboles. De la misma manera que la prueba de hipótesis con un nivel de significancia dado, al introducir comparaciones múltiples, se relaciona directamente con la cantidad de falsos descubrimientos que se pueden encontrar. El problema se describe en el trabajo de David Colquhoun (2014)[8]. Una solución propuesta es la aplicación del método de Benjamini y Hochberg (1995) que garantiza una cota superior para la tasa de falso descubrimiento[9]. A la vez se plantea una comparación con recientes trabajos en controlar el FDR en estructuras de árbol de inferencia[10] y de calcular su efecto[11].

1.3. Objetivos

El objetivo principal de este trabajo es la exploración de nuevos métodos de entrenamiento de árboles de decisión a la vez que evaluar su efectividad en la prevención del “overfitting” en problemas de clasificación y regresión.

A la vez como objetivos secundarios se pretende disminuir el espacio de búsqueda de hiperparámetros. Esto es reducir todos los hiperparámetros relacionados con el crecimiento de los árboles se reduzcan a uno solo. Surge de la experiencia que gran parte del tiempo de análisis e implementación en producción de este tipo de algoritmos se insume en la optimización del modelo, traducido en búsqueda de hiperparámetros, que muchas veces puede resultar en tiempos del orden de días[12]. Como ejemplo de aplicación, en la nube de Google, una máquina virtual de 12 núcleos N2 y 256Gb de RAM para un conjunto de datos de 7 millones de registros y 500 variables puede tardar 12 a 14 horas para 250 iteraciones de búsqueda bayesiana de hiperparámetros.

Este tipo de ciclos en general se puede reducir a:

1. Ingeniería de *features* o variables (*feature engineering*)
2. Optimización de hiperparámetros del algoritmo.
3. Entrenamiento final
4. Prueba de desempeño
5. Despliegue en producción

Los primeros cuatro pasos suelen iterarse un número significativo de veces antes del despliegue, lo que hace muy ventajoso cualquier mejora en sus tiempos de ciclo. En particular la Optimización de hiperparámetros es tiempo de computación donde el analista sólo puede esperar los resultados después de largas corridas mientras que los otros pasos requieren de análisis y son las etapas más importantes. Si se lograra disminuir significativamente el tiempo de optimización de hiperparámetros, se lograría no solo iterar más rápido sino optimizar el tiempo y efectividad del analista del problema.

Para lograr estos objetivos se implementará un nuevo algoritmo de entrenamiento sobre el paquete LightGBM en la forma de un módulo adicional. Se espera que permita su uso con las mismas ventajas de velocidad y manejo eficiente de memoria que el paquete ofrece en otros tipos de entrenadores de árboles de decisión.

Para evaluar la efectividad y desempeño del algoritmo nuevo se realizarán una serie de experimentos donde se entrenará un modelo con los algoritmos actuales y el propuesto. Los entrenamientos se llevarán a cabo sobre conjuntos de datos conocidos e iguales para ambos modelos. Posteriormente se probará su efectividad en nuevos datos a fin de comparar las métricas de desempeño.

2. MATERIALES Y MÉTODOS

2.1. Construcción de árboles de decisión en LightGBM

Los árboles de decisión implementados por LightGBM en su forma más básica crecen por nodos. Es decir se implementa el algoritmo de Árbol de Decisión Seriales pero en lugar de construirlos “por niveles” (ver Figura 2.1), se evalúa cada nodo por separado independientemente del nivel que tenga. En cada iteración se elige un nodo en función de métricas de correlación evaluando todas las variables para los casos que contenga. Este tipo de crecimiento se conoce como “por nodo” (ver Figura 2.2).

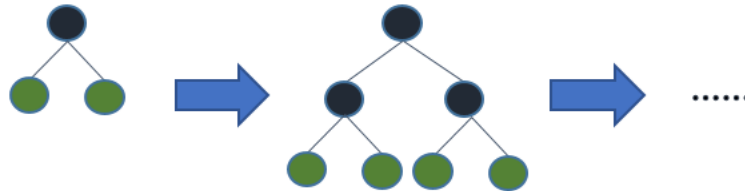


Fig. 2.1: Crecimiento “por niveles”

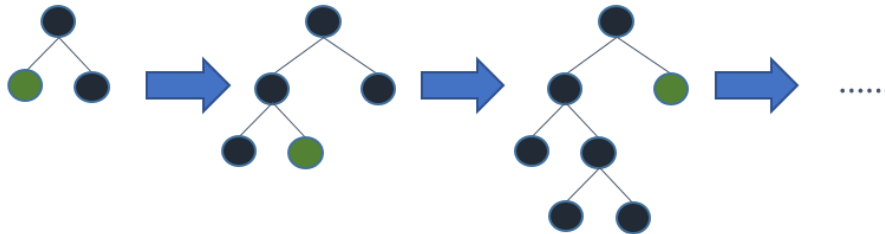


Fig. 2.2: Crecimiento “por nodo”

En la selección de nodos y, dentro de cada nodo, de variables para la división, el algoritmo evalúa para todo el espectro de posibilidades cuales son las mejores variables (las que tienen mayor correlación con la clase) y el punto de corte que maximiza la ganancia de información. El algoritmo elige la máxima ganancia de información en cada hoja para seleccionar la variable y punto de corte. Luego selecciona entre las hojas la que mayor ganancia de información reporta para dividirla.

Luego de cada entrenamiento de árbol se implementa un ensamble de tipo “boosting”: cada árbol trata de predecir los errores (llamados “residuos”) que comete el anterior y se suma su predicción a fin de mejorar el desempeño global.

2.2. Métodos

La hipótesis del presente trabajo es la aplicación de métodos que prevengan el “overfitting” en la construcción de árboles de decisión pero a la vez mantener las ventajas del algoritmo de Gradient Boosting Of Decision Trees. El método se basa en la introducción de ruido al conjunto de datos en la forma de variables nuevas cumpliendo las condiciones siguientes:

1. Se agrega una variable predictora falsa por cada variable del conjunto de datos original.
2. Las distribuciones de los datos en cada variable predictora falsa debe ser la misma que las originales.
3. Las correlaciones entre las variables predictoras deben mantenerse y entre dos variables falsas deben mantenerse las correlaciones con su correspondiente de la misma manera que las originales.

En cada entrenamiento de un árbol se generará el conjunto de variables falsas y cada variable que se determine como candidata para una división, esto es, la generación de una hoja nueva, se comparará con la misma métrica de ganancia de información sobre las variables falsas generadas. Al determinar una variable como más significativa que las variables falsas se procederá a la división, en caso contrario se corta el entrenamiento del árbol.

Como agregado al algoritmo se introducirá un nuevo parámetro que determine la máxima tasa tolerable de descubrimientos falsos. Esto permite que el árbol pueda seguir creciendo si encuentra variables falsas y se corta al llegar a esa tasa.

Con este procedimiento se pretende reemplazar los parámetros de crecimiento de los árboles y del boosting mismo:

1. `min_data_in_leaf`
2. `num_trees`
3. `feature_fraction`
4. `min_gain_to_split`

por un único parámetro que determine el crecimiento de los árboles y no los haga crecer más que lo necesario cuando las variables candidatas sean menos significativas que las variables generadas falsas (`fdr_per_tree`).

Al momento de la comparación se pueden distinguir dos estrategias posibles:

Estrategia de *mejor variable* En esta estrategia se toma la hoja del árbol que de la mayor ganancia de información independientemente de la ganancia que den las variables falsas. Una vez seleccionada esta hoja se compara la ganancia de información que provee contra la mejor de las variables falsas de la misma hoja. Si la

primera es más grande que la segunda se incrementa el contador de descubrimientos verdaderos. En cambio si la ganancia de la mejor variable falsa es mejor que la obtenida con variables reales se incrementa el contador de descubrimientos falsos.

Estrategia de *mejor diferencia* Usando esta estrategia, en cambio, se obtendrá una diferencia entre la ganancia de información de cada hoja para la mejor variable candidata y la mejor variable falsa hoja a hoja. Se tomará la hoja que mayor diferencia tenga como candidata a ser dividida en el árbol de decisión. En caso que esta hora tenga diferencia positiva se incrementa el contador de descubrimientos verdaderos. En cambio si la ganancia de la mejor variable falsa es mejor que la obtenida con variables reales se incrementa el contador de descubrimientos falsos. Esta estrategia tiene la ventaja de intentar encontrar todos los casos verdaderos antes que los correspondientes falsos.

En ambas estrategias la métrica y criterio de parada de entrenamiento de los árboles será la tasa de falso descubrimiento (FDR):

$$FDR = \frac{\text{descubrimientos falsos}}{\text{total de descubrimientos}} < FDR_{max} \quad (2.1)$$

Al encontrar el máximo preestablecido como parámetro para el máximo tolerable de falsos descubrimientos se da por finalizado el entrenamiento y esa iteración del ensamble. Esto da flexibilidad al momento de entrenar un árbol. Se puede ajustar la cantidad de falsos descubrimientos que se toleran a fin de permitir mayor o menor crecimiento, por lo tanto complejidad del modelo, y en consecuencia controlar la tendencia al “overfitting”.

2.3. Datos

Si bien este trabajo no se enfoca en un conjunto de datos concreto sobre el que responder preguntas del fenómeno subyacente, sí se basa en la generación de nuevos métodos para analizarlos y predecir sobre nuevos datos provenientes del mismo fenómeno.

Para ello se compararán los métodos actuales de entrenamiento de árboles (árboles seriales) con el nuevo método (fdr).

El algoritmo se lo implementará como parte de un paquete de Gradient Boosting Machines (LightGBM)[1] para hacer uso de tecnologías recientes en el campo del Machine Learning sobre datos estructurados. Esto permitirá comparar la performance del algoritmo contra el algoritmo base (árboles seriales) y sus hiperparámetros. Con el algoritmo propuesto se pretende eliminar de los hiperparámetros de crecimiento de los árboles controlándolos de manera indirecta con la tasa de descubrimientos falsos.

El algoritmo se debe probar comparando la performance del árbol base en distintos conjuntos de datos con un paso de optimización para ambos siguiendo la metodología siguiente:

-
1. Optimizar el algoritmo base con sus hiperparámetros mediante Optimización Bayesiana.
 2. Optimizar el algoritmo propuesto con sus hiperparámetros mediante Optimización Bayesiana.
 3. Comparar tiempos de optimización, tiempos de entrenamiento, métricas de performance, uso de memoria entre ambos.

Para las comparaciones se pretende utilizar conjuntos de datos estandar: Higgs, Expo, Allstate y otros que suelen usarse como benchmark para este tipo de algoritmos.

3. EXPERIMENTOS PRELIMINARES Y RESULTADOS

Se programó un nuevo módulo sobre el paquete LightGBM que entrena árboles de forma similar al actual *serial* con pequeñas modificaciones:

1. El entrenador debe mantener un estado de variables falsas, esto es el dataset se debería duplicar con una permutación en las filas.
2. Se debe calcular la ganancia de información para todas las variables falsas además de las verdaderas para cada hoja del árbol.
3. Antes de realizar la división de una hoja, se debe verificar que la ganancia máxima (segun la estrategia) sea mayor que la ganancia de la mejor variable falsa en la misma hoja.

En la práctica la permutación se efectúa sobre los resultados de los gradientes que arroja la función de pérdida que utiliza el algoritmo de boosting, por lo que no se guarda la totalidad de los datos, sino una permutación aleatoria de sus índices. Esta optimización permite que la memoria necesaria para un conjunto de datos dado no crezca en demasía ya que solo se requiere mantener un conjunto separado de gradientes y reusar el cache de resultados para un segundo set de datos.

Con el algoritmo implementado como un nuevo entrenador de árboles dentro del paquete LightGBM se han hecho algunos estudios preliminares con datos de ejemplo provistos por el mismo paquete para el problema de clasificación¹.

A ambos entrenadores de árboles se les dio el mismo conjunto de datos, con parámetros de entrenamiento similares, salvo en el caso de los árboles *fdr* donde se dejaron libres los parámetros de crecimiento reemplazándolos por la tasa de descubrimiento falso fija en el 5 %.

Métrica	Serial	FDR (mejor hoja)
AUC (sobre set de validación)	0.815	0.813
Tiempo de entrenamiento (s)	1.227912	0.942676
Cantidad de iteraciones (árboles)	34	89

Tab. 3.1: Resultados preliminares para métricas de performance sobre conjuntos de datos de validación

Los próximos experimentos planteados se realizarán sobre el ciclo de optimización de hiperparámetros:

1. Optimización Bayesiana de hiperparámetros mediante *model-based optimization* (MBO)
2. Entrenamiento del modelo final variando las semillas de forma aleatoria

¹ https://github.com/microsoft/LightGBM/tree/master/examples/binary_classification

3. Comparación de resultados sobre datos nuevos

repetido para el entrenamiento de árboles serial y basados en fdr.

La variación de semillas permite realizar el test de Wilcoxon sobre los resultados obtenidos para estadísticamente garantizar que la mejora es real y no solamente producto del azar.

Este ciclo se probará sobre conjuntos de datos grandes y conocidos como HIGGS², Allstate³ y Expo⁴ de clasificación binaria.

Adicionalmente se intentará inferir la relevancia de otros parámetros del modelo como *learning_rate* y *feature_fraction* que son parte de la lógica de ensamble. Tales parámetros pueden no influir en la calidad del modelo y podrían ser estimados fijos, de esta manera se podría eliminar del espacio de búsqueda de la optimización de hiperparámetros haciendo aún más rápido el ciclo completo de entrenamiento de los modelos.

Para probar esta hipótesis se plantea explorar un espacio con el resto de los hiperparámetros fijos y verificar como influye tanto *learning_rate* como *feature_fraction* en la performance de clasificación.

² <https://archive.ics.uci.edu/ml/datasets/HIGGS>

³ <https://www.kaggle.com/c/ClaimPredictionChallenge>

⁴ <https://community.amstat.org/jointscsg-section/dataexpo/dataexpo2009>

4. TIEMPO ESTIMADO DE TRABAJO

Se estima el tiempo de cada etapa de trabajo sobre la tesis expuesta en el siguiente cuadro

Etapa	Tarea	Tiempo estimado
Etapa 1	Implementación del algoritmo	2 Meses
Etapa 2	Pruebas de concepto	1 Mes
Etapa 3	Pruebas de performance	1 Meses
Etapa 4	Escritura y reporte	2 Meses

Bibliografía

- [1] Microsoft Inc. Lightgbm: A highly efficient gradient boosting decision tree, 2017. <https://github.com/microsoft/LightGBM> consultado el 18/07/2022.
- [2] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree, 2017. *Advances in Neural Information Processing Systems* 30 (NIPS 2017), pp. 3149-3157.
- [3] Emmanuel Candès. Research in the big data era, 9 2016. <https://www.youtube.com/watch?v=aT06nHoo2o0> Emmanuel Candès talks about the reproducibility of scientific research in the Big Data Era and what statistics can offer [Accessed: 2022 07 31].
- [4] Ravid Shwartz-Ziv and Amitai Armon. Tabular data: Deep learning is not all you need. *arXiv:2106.03253v2*, 11 2021.
- [5] Douglas M. Hawkins. The problem of overfitting. *Journal of Chemical Information and Computer Sciences*, 44(1):1–12, 2004. PMID: 14741005.
- [6] John P. A. Ioannidis. Why most published research findings are false. *PLoS Medicine*, 2(8):0696–0701, 2005.
- [7] David D. Jensen and Paul R. Cohen. Multiple comparisons in induction algorithms. *Machine Learning*, (38):309–338, 2000.
- [8] Colquhoun D. An investigation of the false discovery rate and the misinterpretation of p-values. *R. Soc. open sci.*, 1(140216), 2014.
- [9] Benjamini Y. and Hochberg Y. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *J. R. Stat. Soc.*, (57):289–300, 1995.
- [10] Yunxiao Li, Yi-Juan Hu, and Glen A. Statten. A bottom-up approach to testing hypotheses that have a branching tree dependence structure, with false discovery rate control. *arXiv:1903.06850v1*, 3 2019.
- [11] Simeng Shao, Jacob Bien, and Adel Javanmard. Controlling the false split rate in tree-based aggregation. *Data Sciences and Operations, University of Southern California*, 6 2023.
- [12] Dominik Kreuzberger, Niklas Kühl, and Sebastian Hirschl. Machine learning operations (mlops): Overview, definition, and architecture. *IEEE Access*, 11:31866–31879, 2023.